Machine Learning — Unit 1 Exam Answers

Unit 1: Supervised Learning: Classification & Regression (Syllabus Unit III)

SPPU B.E. (AI & DS) 2019 Pattern, Course 417521 · Exam: 70 marks, 2.5 hours · Paper unit = question pair (Q1/Q2 → Unit 1, Q3/Q4 → Unit 2, Q5/Q6 → Unit 3, Q7/Q8 → Unit 4).

Each answer is written for **(X + 2) marks**, where X = highest marks ever asked. Sources cite your PDFs by file name and PDF page number.

1. Unit Snapshot

No.	Question	Max Marks	Years Asked	Importance
U1-1	Explain the K-Nearest Neighbour (KNN) classification algorithm with a suitable example (including numerical).	12	Nov-Dec 2023, Nov-Dec 2024, Nov-Dec 2025	★★★
U1-2	Evaluation measures of binary classification: accuracy, precision, recall, F1-score (formula + numerical on confusion matrix).	12	Nov-Dec 2023, Nov-Dec 2024	★★★
U1-3	What are kernel functions in SVM? Describe Linear, Polynomial, Radial Basis (Gaussian) and Sigmoid kernels.	6	Nov-Dec 2024, Nov-Dec 2025	★★
U1-4	Differentiate between Balanced and Imbalanced classification.	6	Nov-Dec 2024, Nov-Dec 2025	★★
U1-5	Explain the Support Vector Machine (SVM) classification algorithm with a suitable example.	6	Nov-Dec 2023	★
U1-6	Explain the concept of soft margin SVM. How does it differ from hard margin SVM?	6	Nov-Dec 2025	★
U1-7	Explain construction of a multi-class classifier: (i) One-vs-All (ii) One-vs-One.	6	Nov-Dec 2023	★
U1-8	Differentiate between Binary and Multiclass classification.	6	Nov-Dec 2023	★
U1-9	Explain any 4 evaluation measures of Multiclass classification.	4	Nov-Dec 2024	★
U1-10	Explain evaluation measures (SSE, MSE) for a regression model, with numerical.	6	Nov-Dec 2025	★
U1-11	Explain Bayesian Linear Regression.	6	Nov-Dec 2025	★

2. Must-Do First (★★★)

1. **U1-1** — Explain the K-Nearest Neighbour (KNN) classification algorithm with a suitable example (including numerical). (12 M)
2. **U1-2** — Evaluation measures of binary classification: accuracy, precision, recall, F1-score (formula + numerical on confusion matrix). (12 M)

3. Full Answers

U1-1. Explain the K-Nearest Neighbour (KNN) classification algorithm with a suitable example (including numerical).

Importance: ★★★ | **Max asked:** 12 M → **write for 14 M** | **Asked in:** Nov-Dec 2023, Nov-Dec 2024, Nov-Dec 2025

Source: ML Unit 3-6.pdf, pp. 25–28 (Sec. 3.6 K Nearest Neighbor, book pp. 3-25 to 3-28). Worked numerical: solved here from the 2023 paper data.

Asked as: *Apply K-Nearest Neighbor Algorithm (KNN) on following data. Predict the student result for values physics = 6 marks, Chemistry = 8 marks. Consider number of neighbours K = 3 and Euclidean Distance as distance measure.* (Nov-Dec 2023 Q1(a), 12 M) | *Write a detailed note on K Nearest Neighbour algorithm with suitable example.* (Nov-Dec 2024 Q2(c), 8 M) | *Explain K nearest neighbor classification algorithm with suitable example.* (Nov-Dec 2025 Q1(b), 6 M)

Definition

K-Nearest Neighbour (KNN) is a **supervised, non-parametric, instance-based (lazy) learning** algorithm. It classifies a new point by the **majority class of its K closest training points**, measured by a **distance metric** (usually Euclidean).

1. Key Ideas

- **Lazy learner:** no model is built during training; all training data is stored and work is done at prediction time.
- **Non-parametric:** makes no assumption about data distribution.
- **Similarity = closeness:** similar points lie near each other in feature space.
- Used for **classification** (majority vote) and **regression** (average of K neighbours).

2. KNN Algorithm (steps)

1. Load the training data and choose the value of **K** (number of neighbours).
2. For the new (query) point, compute the **distance** to every training point.
3. **Sort** the distances in ascending order.
4. Select the **K nearest** points.
5. **Classification:** assign the **majority class** among the K neighbours. **Regression:** take their **mean** value.
6. Return the predicted class/value.

3. Distance Measures

$$\text{Euclidean: } d(p, q) = \sqrt{[\,(x_1 - x_2)^2 + (y_1 - y_2)^2\,]}$$

$$\text{Manhattan: } d(p, q) = |x_1 - x_2| + |y_1 - y_2|$$

- (x_1, y_1) = query point, (x_2, y_2) = training point.

4. Choosing K

- **Small K** (e.g., 1) → sensitive to **noise**, overfitting.
- **Large K** → smooth boundary but may **underfit**; slow.
- Choose an **odd K** for 2 classes (avoids ties); common rule **K ≈ √n**; tune with cross-validation.

5. Solved Numerical (Nov-Dec 2023, 12 M)

Given: Query → Physics = 6, Chemistry = 8; **K = 3**; Euclidean distance.

Physics (x)	Chemistry (y)	Result	Distance from (6, 8)	Rank
4	3	Fail	$\sqrt{[(6-4)^2 + (8-3)^2]} = \sqrt{29} =$ **5.385**	5
6	7	Pass	$\sqrt{[(0)^2 + (1)^2]} = \sqrt{1} =$ **1.000**	1
7	8	Pass	$\sqrt{[(1)^2 + (0)^2]} = \sqrt{1} =$ **1.000**	1
5	5	Fail	$\sqrt{[(1)^2 + (3)^2]} = \sqrt{10} =$ **3.162**	4
8	8	Pass	$\sqrt{[(2)^2 + (0)^2]} = \sqrt{4} =$ **2.000**	3

- **3 nearest neighbours:** (6,7) Pass – 1.0, (7,8) Pass – 1.0, (8,8) Pass – 2.0
- **Votes:** Pass = 3, Fail = 0
- **Final answer:** the student with Physics = 6, Chemistry = 8 is predicted **PASS**.

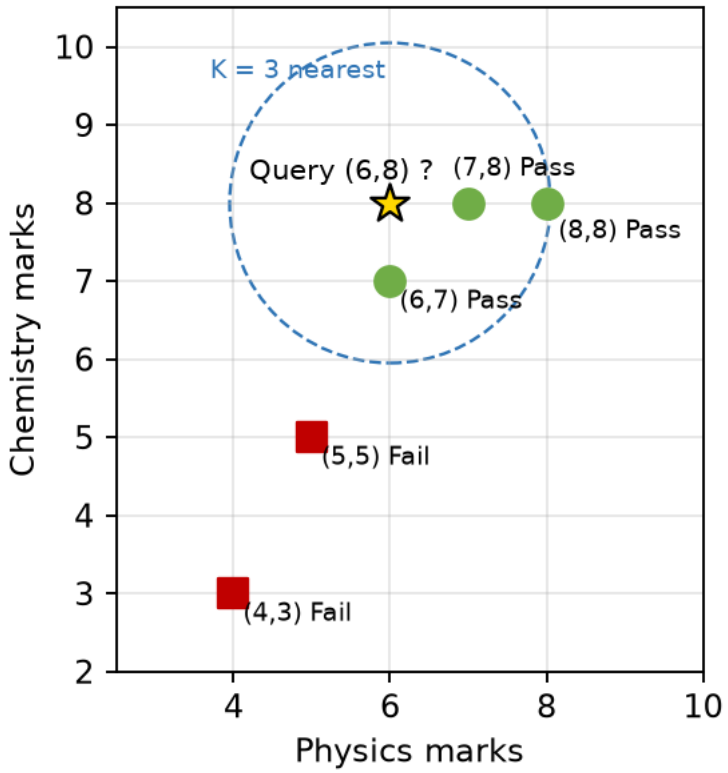


Fig. KNN with K = 3: query (6,8) and its 3 nearest neighbours (all Pass)

- **Axes:** Physics and Chemistry marks (features).
- **Green circles / red squares:** training points labelled Pass / Fail.
- **Star:** query point; **dashed circle:** region holding the K = 3 nearest points.

6. Advantages

- Very **simple**, easy to implement; no training phase.
- Works for **multi-class** problems and naturally adapts to new data.

7. Disadvantages

- **Slow prediction** for large data (distance to all points).
- Needs **feature scaling** (normalisation); sensitive to irrelevant features and noise.
- Suffers from the **curse of dimensionality**; choice of K is critical.

8. Applications

- Recommendation systems, handwriting/digit recognition, medical diagnosis, credit scoring.

Conclusion

KNN predicts by "**asking the neighbours**": compute distances, pick the K closest, take a majority vote. It is simple and effective for small, well-scaled datasets.

Memory trick – "D-S-K-V": Distance → **S**ort → pick **K** → **V**ote. Exam tip: always show the full distance table and the final vote count; state units/labels in the final answer.

U1-2. Evaluation measures of binary classification: accuracy, precision, recall, F1-score (formula + numerical on confusion matrix).

Importance: ★★★ | **Max asked:** 12 M → **write for 14 M** | **Asked in:** Nov-Dec 2023, Nov-Dec 2024

Source: ML Unit 3-6.pdf, pp. 8–16 (Sec. 3.3 Performance Evaluation: confusion matrix, accuracy, precision, recall, F-measure; book pp. 3-8 to 3-16).

Asked as: *Explain any 4 evaluation measures of Binary classification with example?* (Nov-Dec 2023 Q2(a), 6 M) | *Write formula for accuracy, precision, recall and f1-score. Calculate accuracy, precision, recall and f1-score for given example. (Confusion matrix: Predicted Cancer -> 45 actual Cancer, 18 actual No Cancer; Predicted No Cancer -> 12 actual Cancer, 25 actual No Cancer)* (Nov-Dec 2024 Q1(b), 12 M)

Definition

For a **binary classifier**, performance is measured from the **confusion matrix**, a 2 × 2 table that compares **predicted** classes with **actual** classes. Accuracy, precision, recall and F1-score are computed from its four cells.

1. Confusion Matrix

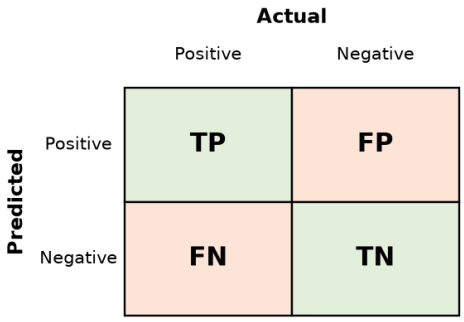


Fig. Confusion matrix for binary classification

- **TP (True Positive):** actual positive, predicted positive.
- **TN (True Negative):** actual negative, predicted negative.
- **FP (False Positive, Type I error):** actual negative, wrongly predicted positive.
- **FN (False Negative, Type II error):** actual positive, wrongly predicted negative.

2. Evaluation Measures (formulas)

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

- Fraction of **all** predictions that are correct. Misleading for **imbalanced** data.

$$\text{Precision} = TP / (TP + FP)$$

- "Of all predicted positives, how many are truly positive?" Important when **FP is costly** (spam filter).

$$\text{Recall (Sensitivity, TPR)} = TP / (TP + FN)$$

- "Of all actual positives, how many did we catch?" Important when **FN is costly** (cancer detection).

$$\text{F1-score} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

- **Harmonic mean** of precision and recall; useful for imbalanced data.

$$\text{Specificity (TNR)} = TN / (TN + FP)$$

- Fraction of actual negatives correctly identified.

$$\text{Error rate} = (FP + FN) / \text{Total} = 1 - \text{Accuracy}\ ;\ \ FPR = FP / (FP + TN)$$

- **ROC curve** plots TPR (recall) vs FPR at different thresholds; **AUC** (area under it) near 1 = good classifier.

3. Solved Numerical (Nov-Dec 2024, 12 M)

Given confusion matrix (rows = predicted, columns = actual, as printed in the paper):

Predicted \ Actual	Cancer (Actual +)	No Cancer (Actual −)
Cancer (Pred +)	**TP = 45**	**FP = 18**
No Cancer (Pred −)	**FN = 12**	**TN = 25**

Total = 45 + 18 + 12 + 25 = **100**

1. **Accuracy** = (45 + 25) / 100 = 70/100 = **0.70 (70 %)**

2. **Precision** = 45 / (45 + 18) = 45/63 = **0.714 (71.4 %)**
3. **Recall** = 45 / (45 + 12) = 45/57 = **0.789 (78.9 %)**
4. **F1-score** = 2 × 0.714 × 0.789 / (0.714 + 0.789) = 1.127 / 1.503 = **0.75 (75 %)**

(check: F1 = 2TP / (2TP + FP + FN) = 90/120 = 0.75 ✓)

5. *(Extra)* **Specificity** = 25 / (25 + 18) = **0.581**; **Error rate** = 30/100 = **0.30**

Interpretation: the model finds 79 % of cancer patients (recall) but 12 patients with cancer are missed (FN) — in medical diagnosis recall should be improved.

4. Example (concept)

- Spam filter: **precision** matters (don't send good mail to spam).
- Disease test: **recall** matters (don't miss a sick patient).

Conclusion

Accuracy gives the overall picture, while precision, recall and F1 show *which kind* of mistakes a classifier makes; for imbalanced problems always report **precision, recall and F1**.

Memory trick: Precision → **P**redicted column/row (TP+FP); **R**ecall → **R**eal (actual) positives (TP+FN). Common mistake: swapping FP and FN — first check whether the table's rows are *actual* or *predicted*.

U1-3. What are kernel functions in SVM? Describe Linear, Polynomial, Radial Basis (Gaussian) and Sigmoid kernels.

Importance: ★★ | **Max asked:** 6 M → **write for 8 M** | **Asked in:** Nov-Dec 2024, Nov-Dec 2025

Source: ML Unit 3-6.pdf, pp. 36–38 (Sec. 3.8 Kernel Functions, 3.8.2 RBF, Gaussian, Polynomial; book pp. 3-36 to 3-38). Note: your textbook writes the polynomial kernel as $(x \cdot x_i + 1)^d$, i.e. $\gamma = 1$, $c = 1$.

Asked as: *What are kernel functions in SVM? Describe the Radial Basis Kernel, and Sigmoid kernel.* (Nov-Dec 2024 Q1(a), 6 M) | *What are kernel functions in SVM? Describe the Radial Basis Kernel, Gaussian, Polynomial, and Sigmoid kernel* (Nov-Dec 2025 Q1(c), 6 M)

Definition

A **kernel function** $K(x_i, x_j)$ computes the **dot product of two points in a higher-dimensional feature space** $\phi(x)$ **without explicitly transforming** them. This **"kernel trick"** lets a linear SVM separate **non-linearly separable** data.

$$K(x_i, x_j) = \varphi(x_i) \cdot \varphi(x_j)$$

1. Why kernels are needed

- Many datasets cannot be separated by a straight line (e.g., circular classes).
- Mapping to a higher dimension makes them **linearly separable**, but explicit mapping is costly.
- The kernel computes the similarity directly → **low cost, same result**.

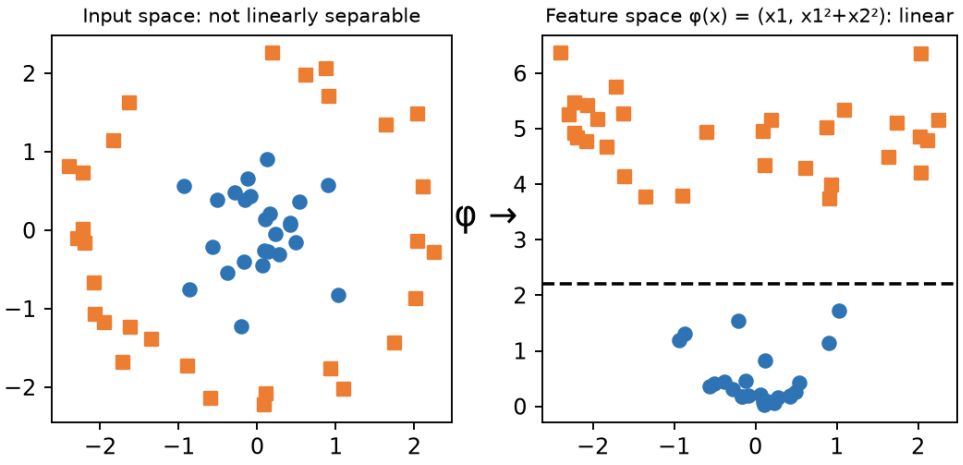


Fig. Kernel trick: circular data becomes linearly separable after mapping φ

- **Left:** input space – inner (blue) and outer (orange) classes; no straight line separates them.
- **Right:** after mapping with ϕ, a straight line (dashed) separates the classes.

2. Types of Kernel Functions

1. **Linear kernel:** K(x, y) = x · y — for linearly separable data, text classification.
2. **Polynomial kernel:** K(x, y) = (γ x · y + c)^d — **d** = degree, **c** = constant; captures feature interactions (image processing).
3. **Radial Basis Function (RBF) / Gaussian kernel:**

$$K(x, y) = \exp(-\|x - y\|^2 / (2\sigma^2)) = \exp(-\gamma \|x - y\|^2)$$

- ‖x − y‖ = Euclidean distance, **σ** = width (spread), **γ = 1/(2σ²)**.
- Value is **1 for identical points** and falls to **0 for far points** → similarity measure.
- Most popular **default** kernel; maps to an **infinite-dimensional** space.
- Large γ → tight, complex boundary (overfit); small γ → smooth boundary.

4. **Sigmoid kernel:**

$$K(x, y) = \tanh(\alpha\, x \cdot y + c)$$

- **α** = slope, **c** = intercept; behaves like a **two-layer perceptron (neural network)** activation.
- Not always positive semi-definite; used less often.

3. Note on "RBF vs Gaussian"

- The **Gaussian kernel is the most common RBF kernel**; any kernel depending only on distance ‖x − y‖ is an RBF. (Laplace RBF: exp(−‖x − y‖/σ).)

Kernel	Formula	Use
Linear	x · y	Linearly separable, text
Polynomial	(γ x·y + c)^d	Interaction features, images
RBF / Gaussian	exp(−γ‖x − y‖²)	General non-linear (default)
Sigmoid	tanh(α x·y + c)	Neural-network-like

Conclusion

Kernels let SVM draw **non-linear boundaries** cheaply; choose **linear** for simple data, **RBF** as a default for non-linear data, and tune γ / degree with cross-validation.

Memory trick – "LPRS": **L**inear, **P**olynomial, **R**BF (Gaussian), **S**igmoid (tanh). RBF = "**R**adius-**B**ased **F**ading similarity".

U1-4. Differentiate between Balanced and Imbalanced classification.

Importance: ★★ | **Max asked:** 6 M → **write for 8 M** | **Asked in:** Nov-Dec 2024, Nov-Dec 2025

Source: ML Unit 3-6.pdf, pp. 3–4 (Sec. 3.1.2 Balanced and Imbalanced Classification Problems; book pp. 3-3, 3-4).

Asked as: *Differentiate Balanced and Imbalanced Classification.* (Nov-Dec 2024 Q2(b), 6 M) | *Differentiate between balanced and imbalanced classification.* (Nov-Dec 2025 Q2(b), 6 M)

Definition

A dataset is **balanced** when all classes have **(nearly) equal** numbers of samples; it is **imbalanced** when one class (**majority**) heavily dominates the other (**minority**). Imbalance is expressed as **majority : minority ratio** (e.g., 8 : 1 : 1).

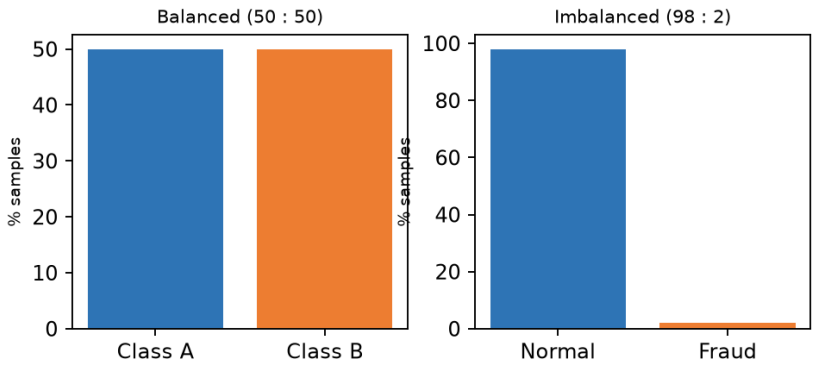


Fig. Balanced vs imbalanced class distribution

Basis	Balanced classification	Imbalanced classification
Class distribution	Classes nearly equal (e.g., 50 : 50)	One class dominates (e.g., 98 : 2)
Model bias	Model learns all classes fairly	Model is **biased toward majority** class
Accuracy as metric	Accuracy is reliable	Accuracy is **misleading** (98 % by always predicting majority)
Preferred metrics	Accuracy, error rate	**Precision, recall, F1, ROC-AUC, PR curve**
Extra handling	Not required	**Resampling** (oversampling, SMOTE, undersampling), **class weights**
Training difficulty	Easier	Harder; minority patterns may be ignored
Decision threshold	Default 0.5 works	Threshold often tuned
Examples	Cat vs dog images (equal), digit recognition	**Fraud detection**, rare disease, spam, fault detection

Example from notes: 100 fruits – 80 oranges, 10 apples, 10 pears → imbalanced, ratio **8 : 1 : 1**.

Conclusion

Balanced data can be modelled directly; imbalanced data needs **resampling or class weighting** and must be judged by **precision/recall/F1**, not accuracy.

Exam tip: Always give one numerical example of imbalance (98 : 2 fraud) and mention **SMOTE** — examiners look for it.

U1-5. Explain the Support Vector Machine (SVM) classification algorithm with a suitable example.

Importance: ★ | **Max asked:** 6 M → **write for 8 M** | **Asked in:** Nov-Dec 2023

Source: ML Unit 3-6.pdf, pp. 29–33 (Sec. 3.7 Linear Support Vector Machines, 3.7.2 Applications, 3.7.3 Limitations; book pp. 3-29 to 3-33).

Asked as: *Explain support Vector Machine classification algorithm with suitable example.* (Nov-Dec 2023 Q1(b), 6 M)

Definition

Support Vector Machine (SVM) is a **supervised** learning algorithm that finds the **optimal hyperplane** which separates classes with the **maximum margin**. The data points closest to the hyperplane are called **support vectors**.

1. Key Terms

- **Hyperplane:** decision boundary $w \cdot x + b = 0$ (a line in 2-D, a plane in 3-D).
- **Margin:** distance between the hyperplane and the nearest points of each class = **2 / ||w||**.
- **Support vectors:** points lying on the margin boundaries; they alone decide the hyperplane.

- **Maximum margin classifier:** larger margin → better generalisation.

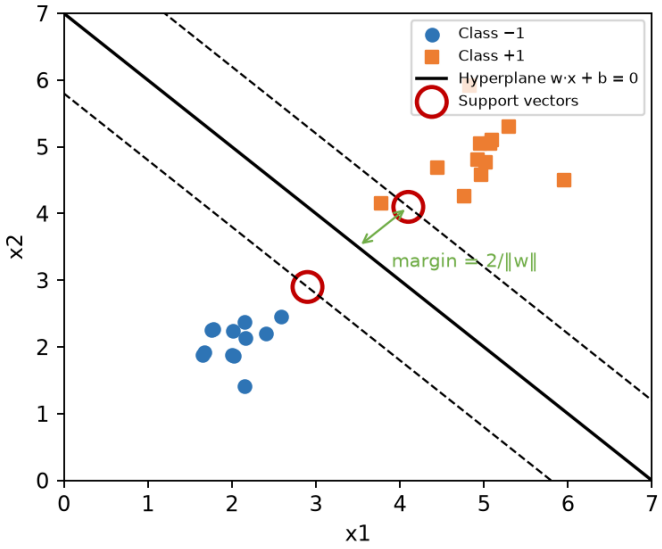


Fig. Linear SVM: hyperplane, margins and support vectors

- **Solid line:** optimal hyperplane $w \cdot x + b = 0$.
- **Dashed lines:** margin boundaries $w \cdot x + b = \pm 1$.
- **Circled points:** support vectors; **green arrow:** margin $2/\|w\|$.

2. Working / Algorithm

1. Take labelled data (x_i, y_i) with $y_i \in \{+1, -1\}$.
2. Find w and b so that every point satisfies $\mathbf{y_i (w \cdot x_i + b) \geq 1}$.
3. **Maximise margin** $2/\|w\| \Leftrightarrow$ **minimise** $\mathbf{\frac{1}{2}\|w\|^2}$ subject to the constraint above.
4. Solve with Lagrange multipliers; points with non-zero multipliers are **support vectors**.
5. Classify new x by $\mathbf{sign(w \cdot x + b)}$.
6. If data is not linearly separable → use **soft margin** (slack) or **kernel trick**.

3. Example

- Email spam: features = frequency of words "free", "offer". SVM finds the widest line separating spam (+1) and not-spam (−1); a new mail is labelled by which side it falls.

4. Advantages / Disadvantages

- **+** Effective in **high-dimensional** spaces; memory-efficient (uses only support vectors); kernels handle non-linear data.
- **−** Slow on very large datasets; choosing kernel and C is tricky; no direct probability output.

Conclusion

SVM is a **maximum-margin classifier** decided only by the support vectors; with kernels it handles complex, non-linear boundaries.

Memory trick: "**Widest street**" — SVM builds the widest street between two classes; the houses on the kerb are the support vectors.

U1-6. Explain the concept of soft margin SVM. How does it differ from hard margin SVM?

Importance: ★ | **Max asked:** 6 M → **write for 8 M** | **Asked in:** Nov-Dec 2025

Source: ML Unit 3-6.pdf, pp. 33–35 (Sec. 3.7 Soft Margin SVM; book pp. 3-33 to 3-35). The term 'hard margin' is not used by name in your textbook; the comparison table is standard.

Asked as: *Explain the concept of a soft margin SVM. How does it differ from a hard margin SVM?* (Nov-Dec 2025 Q2(c), 6 M)

Definition

A **hard-margin SVM** requires **every** training point to be correctly classified and outside the margin. A **soft-margin SVM** allows some points to be **inside the margin or misclassified** by introducing **slack variables ξ_i**, controlled by a penalty **C**.

1. Soft-Margin Formulation

$$\text{Minimise } \tfrac{1}{2}\|w\|^2 + C\,\Sigma\,\xi_i \quad \text{subject to } y_i(w \cdot x_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0$$

- **ξ_i (slack):** 0 → correct & outside margin; 0 < ξ ≤ 1 → inside margin; ξ > 1 → misclassified.
- **C (regularisation):** large C → few violations (narrow margin, risk of overfit); small C → wider margin, more violations.

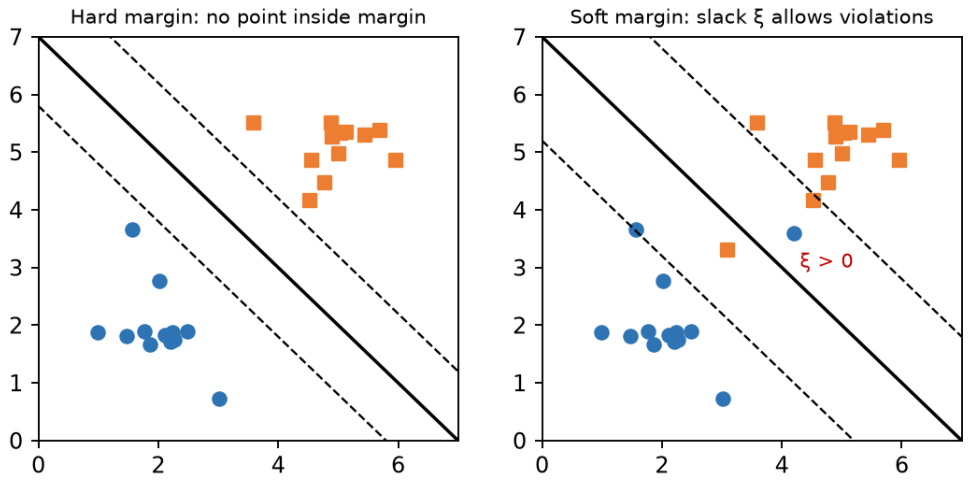


Fig. Hard margin (left) vs soft margin with slack ξ (right)

2. Hard vs Soft Margin

Basis	Hard margin SVM	Soft margin SVM
Data assumption	Perfectly **linearly separable**	Overlapping / noisy data allowed
Misclassification	Not allowed	Allowed via **slack ξ_i**
Objective	min ½‖w‖²	min ½‖w‖² + **C Σξ$_i$**
Constraint	$y_i(w \cdot x_i + b) \geq 1$	$y_i(w \cdot x_i + b) \geq 1 - \xi_i$
Outliers	Very sensitive; may fail	Robust to outliers
Margin width	Narrow	Wider (tunable by C)
Generalisation	Can overfit	Better on real data

Conclusion

Real data is rarely perfectly separable, so **soft margin** (with a tuned C) is used in practice; hard margin is the special case C → ∞.

Memory trick: "**Soft = Slack, C = Strictness**." Big C = strict teacher (few mistakes allowed).

U1-7. Explain construction of a multi-class classifier: (i) One-vs-All (ii) One-vs-One.

Importance: ★ | **Max asked:** 6 M → **write for 8 M** | **Asked in:** Nov-Dec 2023

Source: ML Unit 3-6.pdf, pp. 19–23 (Sec. 3.4 Multiclass Classification, 3.4.2 One-vs-One / One-vs-All; book pp. 3-19 to 3-23).

Asked as: *Explain construction of multi-classifier. i) One Vs. All approach ii) One Vs One approach* (Nov-Dec 2023 Q2(b), 6 M)

Definition

Many algorithms (e.g., SVM, logistic regression) are **binary**. A **multi-class classifier** for N classes is built by **decomposing** the problem into several binary problems using **One-vs-All (OvA / One-vs-Rest)** or **One-vs-One (OvO)**.

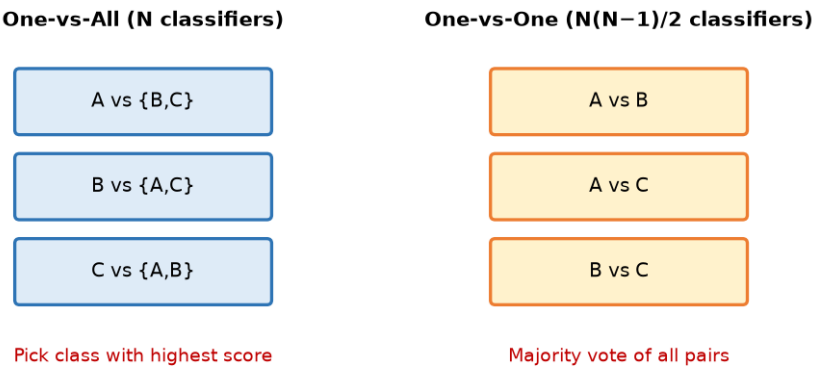


Fig. One-vs-All and One-vs-One for classes A, B, C

1. One-vs-All (One-vs-Rest)

1. For each class i, train one binary classifier: **class i = positive**, **all other classes = negative**.
2. Total classifiers = **N**.
3. For a new sample, run all N classifiers; choose the class whose classifier gives the **highest confidence score**.

- **Example:** classes {Red, Green, Blue} → Red vs (Green, Blue), Green vs (Red, Blue), Blue vs (Red, Green).
- **+** Few classifiers (N). **–** Each classifier sees **imbalanced** data; scores must be comparable.

2. One-vs-One

1. Train a binary classifier for **every pair** of classes.
2. Total classifiers = **N(N – 1)/2**.
3. For a new sample, each classifier votes for one class; the class with **most votes** wins.

- **Example:** 3 classes → 3 classifiers (R vs G, R vs B, G vs B); 4 classes → 6 classifiers.
- **+** Each classifier trains on small, balanced data. **–** Number of classifiers grows quadratically.

OvA	OvO
N classifiers	N(N−1)/2 classifiers
Uses whole dataset per classifier	Uses only 2 classes per classifier
Decision: highest score	Decision: majority vote

Conclusion

OvA is cheaper; OvO is more accurate for algorithms like SVM that scale poorly with data size (sklearn's SVC uses OvO).

Memory trick: OvA = "**one against the crowd**" (N); OvO = "**every pair shakes hands**" (N(N−1)/2).

U1-8. Differentiate between Binary and Multiclass classification.

Importance: ★ | **Max asked:** 6 M → **write for 8 M** | **Asked in:** Nov-Dec 2023

Source: ML Unit 3-6.pdf, p. 3 (Sec. 3.1.1 Binary-vs-Multiclass Classification table; book p. 3-3).

Asked as: *Differentiate between Binary - vs - Multiclass Classification.* (Nov-Dec 2023 Q2(c), 6 M)

Definition

Binary classification assigns each item to one of **two** classes; **multiclass classification** assigns each item to one of **three or more** mutually exclusive classes.

Basis	Binary classification	Multiclass classification
Number of classes	Exactly **2** (0/1, yes/no)	**More than 2** (k classes)
Probability model	Predicts a **Bernoulli** distribution	Predicts a **Multinoulli (categorical)** distribution

Output function	Sigmoid, single output	**Softmax**, one output per class
Typical algorithms	Logistic regression, SVM, KNN, decision tree, Naive Bayes	KNN, decision tree, Naive Bayes, random forest, gradient boosting
Binary algorithms used via	—	**OvA / OvO** decomposition
Evaluation	2×2 confusion matrix, precision, recall, F1	k×k confusion matrix, **per-class / macro / micro** averages
Decision boundary	One boundary	Multiple boundaries
Example	Email **spam or not spam**	Email as **advertisement, newsletter, phishing or personal**; digit 0–9

Conclusion

Binary is the basic two-class case; multiclass generalises it using softmax or binary decomposition, with averaged metrics for evaluation.

Memory trick: Bi = two (bicycle); **Multi** = many — **Bernoulli** for bi, **Multinoulli** for multi.

U1-9. Explain any 4 evaluation measures of Multiclass classification.

Importance: ★ | **Max asked:** 4 M → **write for 6 M** | **Asked in:** Nov-Dec 2024

Source: ML Unit 3-6.pdf, pp. 17–18 and 23–24 (Sec. 3.4 multiclass confusion matrix, per-class precision/recall; Sec. 3.5 Micro-average method; book pp. 3-17, 3-18, 3-23, 3-24).

Definition

For a **multiclass** problem with k classes, a **k × k confusion matrix** is built and metrics are computed **per class** (treating that class as positive and the rest as negative), then **averaged**.

Four Evaluation Measures

1. **Overall accuracy** = Σ diagonal (correct) / total samples.
2. **Per-class precision** $P_i = TP_i / (TP_i + FP_i)$ and **per-class recall** $R_i = TP_i / (TP_i + FN_i)$.
3. **Macro-average** = simple mean of per-class precision/recall/F1 → treats every class **equally**.

$$\text{Macro-P} = (P_1 + P_2 + \ldots + P_k) / k$$

4. **Micro-average** = pool TP, FP, FN of all classes first, then compute → weights by class **frequency**.

$$\text{Micro-P} = \Sigma TP_i / (\Sigma TP_i + \Sigma FP_i)$$

- *(Also: weighted-average F1, Cohen's kappa.)*

Example: 3 classes with precisions 0.9, 0.6, 0.3 → macro-precision = 0.6.

Conclusion

Use **macro** averages when every class is equally important and **micro/weighted** when overall performance on frequent classes matters.

Tip: Macro = "**M**ean of **M**etrics"; Micro = "**M**erge counts first".

U1-10. Explain evaluation measures (SSE, MSE) for a regression model, with numerical.

Importance: ★ | **Max asked:** 6 M → **write for 8 M** | **Asked in:** Nov-Dec 2025

Source: ML Textbook.pdf, pp. 57–58 (Sec. 2.2 Cost Functions (Evaluation Metrics): Mean Error, MSE, MAE, RMSE, R-squared; book pp. 2-22 to 2-25). SSE is the total of the squared-error column in the textbook tables.

Asked as: *Explain evaluation measures (SSE, MSE) for regression model. For a given data having 100 examples, if squared errors SE1, SE2, and SE3 are 13.33, 3.33 and 4.00 respectively, calculate Mean Squared Error (MSE). State the formula for MSE.* (Nov-Dec 2025 Q1(a), 6 M)

Definition

Regression models predict **continuous values**; their error is measured from the **residual $e_i = y_i - \hat{y}_i$** (actual − predicted). **SSE** and **MSE** are the basic squared-error measures.

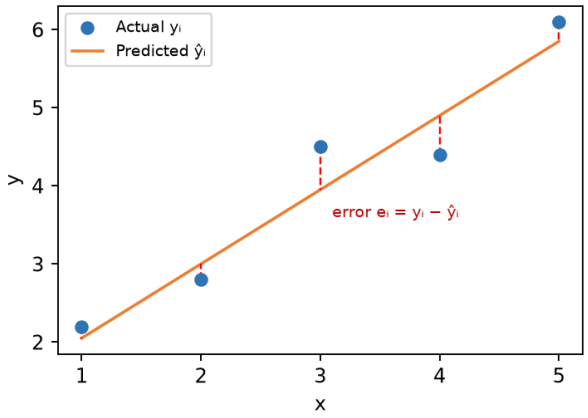


Fig. Residuals (red dashed) between actual points and the regression line

1. Formulas

$$SSE = \Sigma\,(y_i - \hat{y}_i)^2 \quad (i = 1 \ldots n)$$

$$MSE = SSE / n = (1/n)\,\Sigma\,(y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{MSE}\,, \quad MAE = (1/n)\,\Sigma\,|y_i - \hat{y}_i|$$

- y_i = actual value, $\hat{y}_i$ = predicted value, **n** = number of examples.
- Squaring removes sign and **penalises large errors** more. Lower = better.
- Your textbook calls these **cost functions (evaluation metrics)** and also lists **Mean Error (ME)**, which is weak because positive and negative errors cancel.
- **Textbook example:** actual (10, 8, 5, 7, 4) vs predicted (9.2, 8.3, 4.7, 7.9, 3.1): squared errors 0.64, 0.09, 0.09, 0.81, 0.81 → **SSE = 2.44**, **MSE = 2.44/5 = 0.488**, **RMSE = 0.698**.

2. Numerical (Nov-Dec 2025)

Given: n = 100 examples; squared-error terms $SE_1 = 13.33$, $SE_2 = 3.33$, $SE_3 = 4.00$.

1. **SSE** = 13.33 + 3.33 + 4.00 = **20.66**
2. **MSE** = SSE / n = 20.66 / 100 = **0.2066**
3. *(If the three values are SSEs of three separate models, $MSE_1 = 0.1333$, $MSE_2 = 0.0333$, $MSE_3 = 0.04$ → model 2 is best.)*

Conclusion

SSE measures total squared error, MSE the average squared error per example; both should be **minimised** and are used to compare regression models.

Tip: Always write the formula first, then substitute — formula carries marks even if arithmetic slips.

U1-11. Explain Bayesian Linear Regression.

Importance: ★ | **Max asked:** 6 M → **write for 8 M** | **Asked in:** Nov-Dec 2025

Source: ML Textbook.pdf, pp. 56–57 (Sec. 2.1.13 Bayesian Linear Regression; book pp. 2-21, 2-22).

Asked as: *Explain Baysian Linear Regression.* (Nov-Dec 2025 Q2(a), 6 M)

Definition

Bayesian Linear Regression (BLR) treats the regression weights **w** as **random variables with a probability distribution** rather than fixed numbers. Using **Bayes' theorem**, a **prior** belief about w is updated with data to obtain a **posterior** distribution, giving predictions **with uncertainty**.

1. Model

$$y = w^T x + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2)$$

$$\text{Posterior } P(w \mid D) = P(D \mid w) \cdot P(w) \,/\, P(D) \quad \propto \text{ Likelihood} \times \text{Prior}$$

- **P(w)** = prior (e.g., Gaussian $N(0, \alpha^{-1}I)$); **P(D | w)** = likelihood of data; **P(w | D)** = posterior.

2. Steps

1. Choose a **prior** distribution over weights (usually Gaussian).
2. Define the **likelihood** of the observed data given weights (Gaussian noise).
3. Compute the **posterior** = likelihood × prior (Gaussian prior + Gaussian likelihood → Gaussian posterior).
4. **Predict** by averaging over all weights → predictive mean **and variance**.

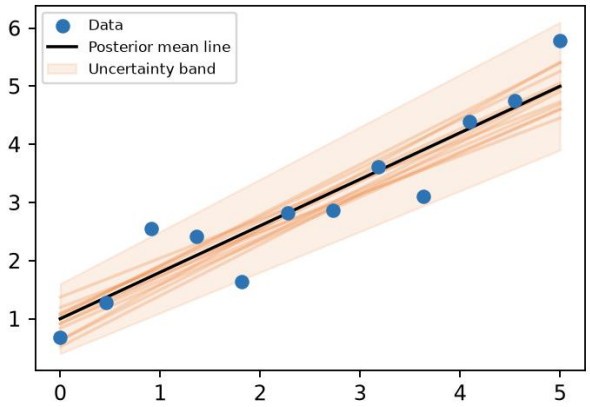


Fig. BLR gives a family of plausible lines and an uncertainty band

3. Advantages / Disadvantages

- **+** Gives **uncertainty (confidence)** of predictions; works with **small data**; prior acts as **regularisation** (MAP with Gaussian prior = Ridge regression).
- **−** Computationally heavier; result depends on choice of prior.

Example

Predicting house price from area with only 10 houses: BLR predicts ₹50 L **± ₹4 L**, while ordinary regression gives only ₹50 L.

Conclusion

BLR = linear regression + Bayes' theorem; it outputs a **distribution** of predictions instead of a single value.

Memory trick: "**Prior + Data = Posterior**" (PDP).

4. Last-Minute Revision Sheet

Key Definitions

- **KNN:** lazy, non-parametric; majority vote of K nearest (Euclidean) neighbours.
- **SVM:** maximum-margin hyperplane; **support vectors** decide it; margin = $2/\|w\|$.
- **Soft margin:** slack ξ_i + penalty C; hard margin = no violations.
- **Kernel trick:** $K(x, y) = \phi(x) \cdot \phi(y)$ without computing ϕ.
- **Imbalanced data:** majority dominates → use precision/recall/F1, SMOTE, class weights.
- **OvA:** N classifiers; **OvO:** $N(N-1)/2$ classifiers + voting.
- **BLR:** weights have prior → posterior via Bayes.

Formulas

Measure	Formula
Euclidean distance	$\sqrt{\Sigma(x_i - y_i)^2}$
Accuracy	(TP + TN) / Total
Precision	TP / (TP + FP)
Recall	TP / (TP + FN)
F1	2PR / (P + R) = 2TP / (2TP + FP + FN)
Specificity	TN / (TN + FP)
RBF kernel	$\exp(-\gamma\|x - y\|^2)$, $\gamma = 1/2\sigma^2$
Polynomial kernel	$(\gamma\, x \cdot y + c)^d$
Sigmoid kernel	$\tanh(\alpha\, x \cdot y + c)$
Soft-margin SVM	$\min \tfrac{1}{2}\|w\|^2 + C\Sigma\xi_i$
SSE / MSE	$\Sigma(y - \hat{y})^2$; SSE/n

Memory Tricks

- KNN = **D-S-K-V** (Distance, Sort, K, Vote).
- Kernels = **LPRS**. SVM = "widest street".
- **P**recision → **P**redicted positives; **R**ecall → **R**eal positives.
- Soft margin = **S**lack, C = **S**trictness.
- Numericals: draw the table, write formula, substitute, box the answer.

5. Assumptions & Gaps

- **Unit mapping:** papers do not label units; Q1/Q2 are taken as Unit 1 (= syllabus Unit III, Classification) because every paper places classification/regression evaluation here.
- **2024 Q1(b) confusion matrix:** the printed table has *Actual* values as columns and *Predicted* values as rows; it was read as TP = 45, FP = 18, FN = 12, TN = 25. If your teacher reads rows as actual, FP and FN swap: precision = 45/57 = 0.789, recall = 45/63 = 0.714, F1 = 0.75 and accuracy = 0.70 (unchanged).
- **2025 Q1(a) MSE:** the wording "100 examples, squared errors SE1, SE2, SE3" is ambiguous; main answer sums them (SSE = 20.66, MSE = 0.2066) and the alternative reading is shown.
- All Unit 1 topics were found in your textbooks (Classification in ML Unit 3-6.pdf; regression metrics and Bayesian Linear Regression in ML Textbook.pdf, Unit II). No ⚠ flags. Only the name 'hard margin' does not appear in the OCR text of the textbook.